Department of Computer Science & Engineering
The University of Texas at Arlington

Architectural Design Specification
CSE 4316: Senior Design I
Spring 2026

AgriHome project logo: leaf and camera lens motif for vision-based agricultural monitoring.

AgriHome

GOVINDA AWASTHI
LAXMAN BHUSHAL
THUC DANG
SAGAR KHADKA
MUHAMMAD SAAD MEMON
ALEX TABLIZO
NATHAN NGUYEN TRAN

Revision History

Revision	Date	Author(s)	Description
0.1	03.04.2026	Team	Document created; initial architecture draft
1.0	03.04.2026	Team	Layered overview, figures, PDF verified
1.1	03.04.2026	Team	Raised system-level abstraction in overview sections
2.0	03.04.2026	GA, LB, TD, SK, MSM, AT, NNT	Restructure to CSE 4316 ADS format; subsystems, interfaces; team roster

Contents

1 Introduction **5**
- 1.1 Purpose . . . 5
- 1.2 Scope . . . 5
- 1.3 System summary . . . 5
- 1.4 Document organization . . . 6

2 Project Overview **7**
- 2.1 Presentation layer description . . . 7
- 2.2 Application layer description . . . 7
- 2.3 Processing layer description . . . 8
- 2.4 Data layer description . . . 8
- 2.5 Layered architecture figure . . . 9
- 2.6 System workflow overview . . . 9

3 System Description **10**
- 3.1 Presentation layer composition . . . 10
- 3.2 Application layer composition . . . 10
- 3.3 Processing layer composition . . . 11
- 3.4 Data layer composition . . . 11
- 3.5 End-to-end data flow summary . . . 11

4 Data Layer Subsystems **12**
- 4.1 Write coordination subsystem . . . 12
  - 4.1.1 Assumptions . . . 12
  - 4.1.2 Responsibilities . . . 12
  - 4.1.3 Subsystem interfaces . . . 12
- 4.2 Integrity and constraint enforcement subsystem . . . 12
  - 4.2.1 Assumptions . . . 13
  - 4.2.2 Responsibilities . . . 13
  - 4.2.3 Subsystem interfaces . . . 13
- 4.3 Query and transaction execution subsystem . . . 13
  - 4.3.1 Assumptions . . . 13
  - 4.3.2 Responsibilities . . . 13
  - 4.3.3 Subsystem interfaces . . . 14
- 4.4 PostgreSQL domain database subsystem . . . 14
  - 4.4.1 Assumptions . . . 14
  - 4.4.2 Responsibilities . . . 14
  - 4.4.3 Subsystem interfaces . . . 14
- 4.5 Image and media storage subsystem . . . 14
  - 4.5.1 Assumptions . . . 15
  - 4.5.2 Responsibilities . . . 15
  - 4.5.3 Subsystem interfaces . . . 15

5 Presentation Layer Subsystems **16**
- 5.1 Authentication and session UI subsystem . . . 16
  - 5.1.1 Assumptions . . . 16

5.1.2 Responsibilities . . . 16
5.1.3 Subsystem interfaces . . . 16
5.2 Dashboard and overview subsystem . . . 16
5.2.1 Assumptions . . . 16
5.2.2 Responsibilities . . . 17
5.2.3 Subsystem interfaces . . . 17
5.3 Tray monitoring console subsystem . . . 17
5.3.1 Assumptions . . . 17
5.3.2 Responsibilities . . . 17
5.3.3 Subsystem interfaces . . . 17
5.4 Plant detail and health history subsystem . . . 17
5.4.1 Assumptions . . . 18
5.4.2 Responsibilities . . . 18
5.4.3 Subsystem interfaces . . . 18
5.5 Mesh and topology subsystem . . . 18
5.5.1 Assumptions . . . 18
5.5.2 Responsibilities . . . 18
5.5.3 Subsystem interfaces . . . 18
5.6 Schedule management subsystem . . . 19
5.6.1 Assumptions . . . 19
5.6.2 Responsibilities . . . 19
5.6.3 Subsystem interfaces . . . 19

6 Application Layer Subsystems **20**
6.1 API entry and routing subsystem . . . 20
6.1.1 Assumptions . . . 20
6.1.2 Responsibilities . . . 20
6.1.3 Subsystem interfaces . . . 20
6.2 Session and authentication broker subsystem . . . 20
6.2.1 Assumptions . . . 21
6.2.2 Responsibilities . . . 21
6.2.3 Subsystem interfaces . . . 21
6.3 Input validation subsystem . . . 21
6.3.1 Assumptions . . . 21
6.3.2 Responsibilities . . . 21
6.3.3 Subsystem interfaces . . . 21
6.4 Domain orchestration subsystem . . . 22
6.4.1 Assumptions . . . 22
6.4.2 Responsibilities . . . 22
6.4.3 Subsystem interfaces . . . 22
6.5 GraphQL query and mutation facade subsystem . . . 22
6.5.1 Assumptions . . . 22
6.5.2 Responsibilities . . . 22
6.5.3 Subsystem interfaces . . . 23
6.6 Response and error handling subsystem . . . 23
6.6.1 Assumptions . . . 23
6.6.2 Responsibilities . . . 23
6.6.3 Subsystem interfaces . . . 23

7 Processing Layer Subsystems **24**
7.1 Species and leaf inference adapter subsystem 24
7.1.1 Assumptions 24
7.1.2 Responsibilities 24
7.1.3 Subsystem interfaces 24
7.2 Tray vision and layout adapter subsystem 24
7.2.1 Assumptions 25
7.2.2 Responsibilities 25
7.2.3 Subsystem interfaces 25
7.3 Inference normalization subsystem 25
7.3.1 Assumptions 25
7.3.2 Responsibilities 25
7.3.3 Subsystem interfaces 25

References **26**

List of Figures

1 AgriHome Vision Console: four-layer architecture and trust boundaries. 9
2 Subsystem definitions and primary vertical data flow (application also reads/writes the data layer directly for most CRUD paths). 10
3 Write coordination subsystem: accepts write intents and forwards them toward integrity checks. 12
4 Integrity subsystem: validates operations before execution. 12
5 Query executor: issues SQL and coordinates binary I/O with storage. 13
6 PostgreSQL domain database subsystem. 14
7 Image and media storage subsystem and relationship to application reads. 15
8 Authentication UI: user actions become authenticated API calls. 16
9 Dashboard subsystem requests summary aggregates from the application tier. 16
10 Tray console: drives reads and optional vision requests through the application layer. . 17
11 Plant detail subsystem: CRUD and reporting views. 18
12 Mesh subsystem interacts with mesh and member tray aggregates. 18
13 Schedule subsystem persists policies via application APIs. 19
14 API entry: first server hop after the presentation layer. 20
15 Session broker: establishes trusted account context for downstream work. 20
16 Validation subsystem: rejects bad input early. 21
17 Domain orchestration: central business logic and integration point. 22
18 GraphQL facade: thin mapping to domain services. 22
19 Response handler: maps internal results to HTTP/GraphQL responses. 23
20 Species inference adapter: hosted model runtime may be external. 24
21 Tray vision adapter: round-trip inference for tray imagery. 24
22 Normalization: single canonical shape for downstream persistence. 25

List of Tables

2 Write coordination subsystem interfaces 12
3 Integrity and constraint enforcement subsystem interfaces 13
4 Query and transaction execution subsystem interfaces 14
5 PostgreSQL domain database subsystem interfaces 14
6 Image and media storage subsystem interfaces 15
7 Authentication and session UI subsystem interfaces 16
8 Dashboard and overview subsystem interfaces 17
9 Tray monitoring console subsystem interfaces 17
10 Plant detail and health history subsystem interfaces 18
11 Mesh and topology subsystem interfaces 18
12 Schedule management subsystem interfaces 19
13 API entry and routing subsystem interfaces 20
14 Session and authentication broker subsystem interfaces 21
15 Input validation subsystem interfaces 21
16 Domain orchestration subsystem interfaces 22
17 GraphQL facade subsystem interfaces 23
18 Response and error handling subsystem interfaces 23
19 Species and leaf inference adapter interfaces 24
20 Tray vision and layout adapter interfaces 25
21 Inference normalization subsystem interfaces 25

1 Introduction

1.1 Purpose

The **AgriHome Vision Console** is a web-based monitoring and decision-support product for controlled-environment agriculture and laboratory plant studies. Operators and researchers use it to observe tray-level imagery, track individual plants, review automated health assessments, organize trays into logical **meshes**, and manage image-capture schedules. This **Architectural Design Specification (ADS)** records how the product is decomposed into layers and subsystems, how data moves among them, and which interfaces connect internal components to each other and to external partners.

This document is a companion to the team's **System Requirements Specification (SRS)** for the same project.[1] The SRS states *what* the system must do for customers (for example, plant health presentation, automatic capture, environmental data, dashboards, authentication, and secure storage). This ADS states *how* those responsibilities are allocated to software structure: presentation, application orchestration, processing (vision and analytics adapters), and persistent data.

1.2 Scope

In scope for this ADS are: the logical four-layer architecture; subsystem names, assumptions, responsibilities, and interface summaries; major data flows from user action to storage; identification of optional processing and search backends; and deployment context at the level of application, database, and object storage.

Out of scope are: user-interface visual design mockups beyond structural responsibilities; detailed REST or GraphQL field specifications; source file paths and build commands (covered in engineering documentation); formal threat models; and training procedures for ML models.

1.3 System summary

AgriHome addresses limitations of ad hoc spreadsheets, disconnected camera feeds, and manual record-keeping by providing a **single console** that ties together:

- **Identity and access** — authenticated users interact only with their permitted trays, plants, and meshes.
- **Observation** — latest captures, charts, and monitoring history support the requirements for plant status display, historical data, and dashboard-style summaries.
- **Interpretation** — health reports and optional model-backed inference enrich raw imagery (where configured).
- **Planning** — schedules express when captures or checks should occur, supporting automatic image capture policies described in the SRS.
- **Persistence** — relational storage holds authoritative domain state; object storage holds images and large artifacts referenced from that state.

The reference implementation uses a modern full-stack web platform (browser and installable PWA client, server-hosted application, PostgreSQL, file storage, Firebase Authentication, and optional HTTP-based inference services). The architecture in this document is described so that those technologies could be substituted behind the same layer boundaries where a future maintainer requires it.

1.4 Document organization

Section 2 presents the **project overview** and describes the four primary layers. Section 3 gives a **system description** with a consolidated subsystem-and-data-flow view. Sections 4–7 document **data**, **presentation**, **application**, and **processing** layer subsystems respectively, each with assumptions, responsibilities, and interface tables. The **References** section lists external citations and the companion SRS.

2 Project Overview

The AgriHome Vision Console is designed using a **layered architecture**. Each layer has a distinct role; layers interact through well-defined interfaces (HTTP APIs between client and server, internal service calls within the server, SQL and file I/O toward stores, and optional outbound calls to inference services). This separation improves modularity, testability, and the ability to scale or replace one concern (for example, swapping a vision provider) without rewriting unrelated areas.

The system comprises **four primary layers**:

1. **Presentation Layer** — user-facing experience in the browser or PWA: landing and marketing entry, authentication, dashboard, tray and plant views, mesh management, schedules, charts, and uploads.

2. **Application Layer** — server-side control: HTTP and GraphQL entry points, session verification, validation, routing of work to domain services, coordination of reads and writes to the data layer, and invocation of the processing layer when needed.

3. **Processing Layer** — computer-vision and analytics *adapters*: optional services that accept images or features and return structured labels, detections, or scores for the application layer to persist or display. This layer may run on separate infrastructure (for example, GPU hosts).

4. **Data Layer** — durable storage: relational database for domain entities (trays, plants, meshes, schedules, reports, events, predictions), and object or file storage for imagery; optional vector index for similarity search when enabled.

2.1 Presentation layer description

The Presentation Layer is the primary interface between **people** and the product. It implements responsive layouts for desktop and mobile, supports installable PWA behavior for field-adjacent use, and communicates with the Application Layer exclusively through HTTPS (including authenticated cookie-based sessions after sign-in).

Users register or sign in, navigate trays and plants, upload photos, trigger or view vision-assisted results, inspect monitoring history, configure schedules, and manage meshes. The layer is responsible for presenting SRS-mandated capabilities such as health status, historical trends, dashboard summaries, and settings-oriented flows at a user-experience level; it does not embed direct database access.

Key responsibilities:

- Capture user input (forms, file selection, navigation).
- Render charts, tables, imagery, and status text clearly and accessibly.
- Invoke application APIs and display responses or structured errors.
- Encapsulate client-only concerns (progressive enhancement, local caching of shell assets where applicable).

2.2 Application layer description

The Application Layer is the **central coordinator**. It receives every authenticated request from the Presentation Layer, interprets it, enforces authorization tied to the authenticated identity, validates payloads, delegates to domain-oriented modules (tray and plant lifecycle, mesh topology, schedules, monitoring logs, file references), and returns responses.

It is also responsible for **bridging** to the Processing Layer: when a workflow requires species classification, tray-level detection, or similar inference, the application packages the request, calls the appropriate adapter, interprets the result, and maps it into domain records. It never exposes raw database credentials to clients.

Key responsibilities:

- Expose a stable API surface for the console and future integrators.
- Maintain session integrity and ownership rules (which user owns which trays).
- Orchestrate transactions across relational updates and file metadata.
- Isolate failures in optional backends so core monitoring remains usable when possible.

2.3 Processing layer description

The Processing Layer performs **compute-intensive or model-based** work that should not block the interactive path of the UI thread on the server. In AgriHome this includes, when configured, **species or leaf-level classification**, **tray-level vision** (for example plant counting or bounding regions), and any future scoring models. These functions may be implemented as separate HTTP services maintained by the same team or by an ML operations group.

When no external model is available, the Application Layer may apply **deterministic or simulated** behavior for development and demonstration; the architecture still treats that path as "processing" logically because it produces structured inference output distinct from raw CRUD.

Key responsibilities:

- Accept image or feature input in a contract agreed with the application tier.
- Run detection, embedding, or classification pipelines as implemented.
- Return normalized results (labels, confidence, geometry) to the Application Layer.
- Remain stateless with respect to long-term domain truth; persistence is always via the application and data layers.

2.4 Data layer description

The Data Layer is the **authoritative foundation** for AgriHome. The relational store holds entities and relationships that satisfy SRS concepts: plant health records over time, capture references, environmental or monitoring events as modeled, mesh membership, and schedules. Object storage holds binary assets (camera frames, user uploads) referenced by identifiers or URLs in the relational model.

Optional components such as a **vector database** participate in this layer when similarity or embedding search is required; they remain secondary to the relational system of record.

Key responsibilities:

- Persist and retrieve structured domain data with consistency expectations documented per operation.
- Store and stream large binaries efficiently.
- Enforce integrity through schema, constraints, and application-side validation.
- Support backup, migration, and operational monitoring consistent with deployment policy.

2.5 Layered architecture figure

Figure 1 summarizes the four layers and primary external actors. Companion diagrams in Section 3 refine internal subsystems and data flow.

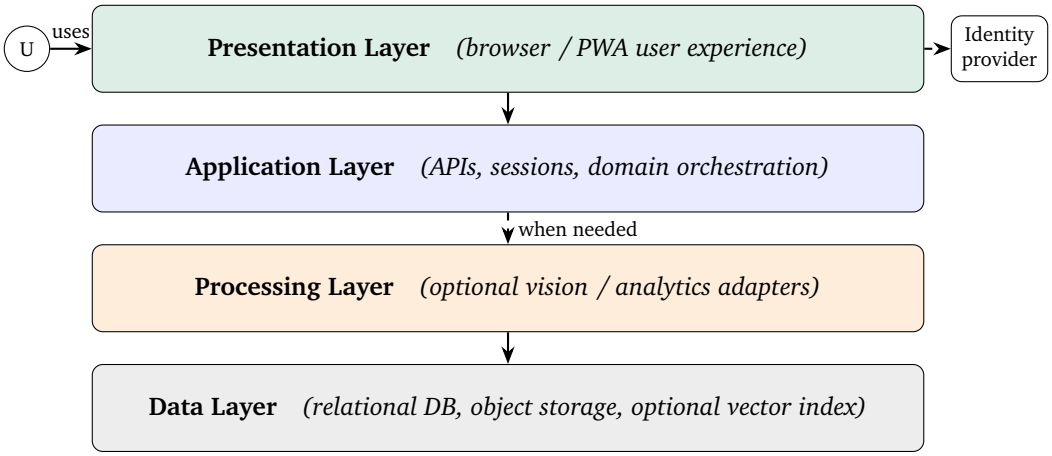

Figure 1: AgriHome Vision Console: four-layer architecture and trust boundaries.

2.6 System workflow overview

A typical end-to-end path begins when a signed-in user opens a **tray** or **plant** view in the Presentation Layer. The client requests data from the Application Layer, which reads the **Data Layer** (relational and, when needed, media URLs). When the user uploads an image or requests vision-assisted analysis, the Application Layer may forward pixels or references to the **Processing Layer**; results return as structured data, are validated, and are written to the database and/or file store. The updated state flows back to the Presentation Layer for display, satisfying SRS expectations for visible health status, history, and dashboards.

3 System Description

This section provides a consolidated **subsystem definition** and **data-flow** narrative for the AgriHome Vision Console. Figure 2 maps major building blocks to the four layers introduced in Section 2. The text following the figure walks through each layer and explains how information moves from user intent to durable storage and back.

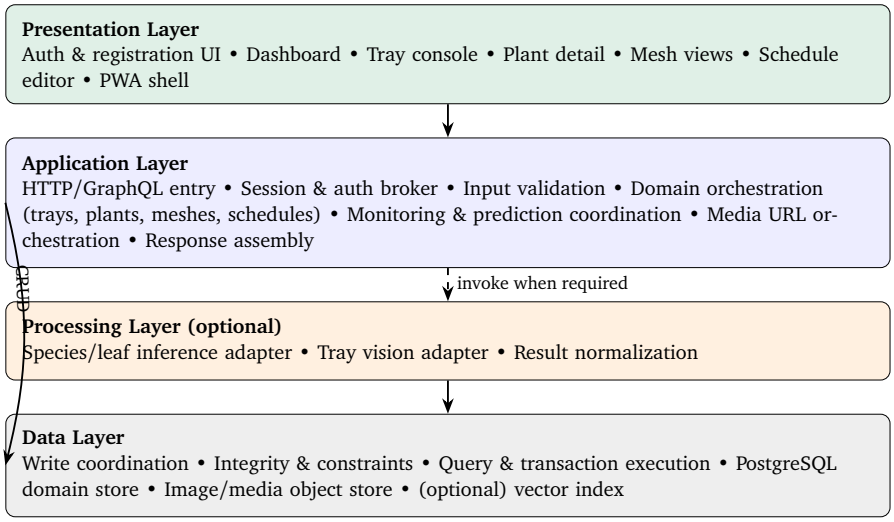

Figure 2: Subsystem definitions and primary vertical data flow (application also reads/writes the data layer directly for most CRUD paths).

3.1 Presentation layer composition

The Presentation Layer is organized around **workspaces** that match operator mental models. **Authentication and registration** establish identity and tie the browser to a server session. The **dashboard** summarizes tray health and recent activity for SRS-aligned "at a glance" monitoring. The **tray console** shows imagery, monitoring charts, and plant rows for a single tray. **Plant detail** focuses on one organism: photos, editable metadata, report history, and plant-scoped logs. **Mesh views** aggregate multiple trays for experiments or facility zones. The **schedule editor** binds capture policy to trays or meshes. The **PWA shell** supplies offline-friendly installation and static asset caching where supported.

User gestures in these areas become HTTPS requests to the Application Layer; no presentation component queries the database directly.

3.2 Application layer composition

The Application Layer terminates TLS, verifies cookies or tokens, and maps the caller to an **account context**. **Validation** rejects malformed payloads before any side effect. **Domain orchestration** implements use cases: create or update plants, attach photos, record monitoring events, link trays into meshes, upsert schedules, and fetch history for charts. When a use case requires inference, the same layer forwards work to the Processing Layer and merges outcomes into domain writes. **Response assembly** maps internal results to JSON or GraphQL shapes consumed by the UI.

This layer is the only tier that routinely touches both the Data Layer and (optionally) the Processing Layer.

3.3 Processing layer composition

The Processing Layer is deliberately **thin at the product boundary**: it exposes inference contracts, not user workflows. A **species or leaf adapter** may classify a single-plant photo for onboarding flows. A **tray vision adapter** may estimate plant count or regions on a tray image. **Result normalization** ensures that bounding boxes, labels, and scores share a consistent schema before the Application Layer persists them.

If adapters are offline, the Application Layer may return errors or deterministic stand-ins according to deployment policy; the SRS still governs what the customer should observe in those cases.

3.4 Data layer composition

The Data Layer splits **transactional** concerns from **bulk media**. A **write coordination** path sequences inserts and updates so that relational rows and file metadata stay aligned. **Integrity and constraints** guard required fields, foreign keys, and business invariants. **Query and transaction execution** runs SELECT/INSERT/UPDATE/DELETE operations against **PostgreSQL**. The **object store** retains images and large binaries; the relational model stores pointers or paths. An optional **vector index** supports similarity workflows without becoming the system of record.

3.5 End-to-end data flow summary

The data flow begins when a user interacts with the Presentation Layer. Requests pass to the Application Layer, which authorizes and validates them. For ordinary reads, the Application Layer queries the Data Layer and returns results. For vision-assisted actions, the Application Layer calls the Processing Layer, then commits structured outputs through the Data Layer. Responses propagate back to the Presentation Layer for rendering, completing the loop required by the SRS for health display, history, and dashboard experiences.

4 Data Layer Subsystems

The Data Layer provides durable storage for all AgriHome domain state and media. It is structured into subsystems that mirror the reference senior-design pattern: coordinated writes, integrity checks, query execution, the primary relational database, and dedicated image storage.

4.1 Write coordination subsystem

The **write coordination** subsystem sequences operations that must stay consistent across multiple rows or across database rows and file metadata (for example, creating a plant and linking an uploaded photo). It receives structured write intents from the Application Layer and orders them so partial failures can be detected and reported.

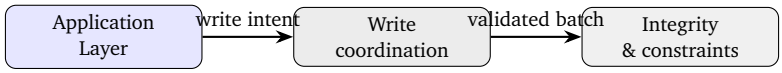

Figure 3: Write coordination subsystem: accepts write intents and forwards them toward integrity checks.

4.1.1 Assumptions

- The Application Layer supplies writes in transactional units appropriate to each use case.
- Concurrent writers may exist; isolation expectations are defined per operation.
- Object storage availability may differ from database availability; coordination must surface errors clearly.

4.1.2 Responsibilities

- Accept batched write intents from domain services.
- Order dependent operations (metadata before binary commit or the reverse, per policy).
- Expose success/failure to the Application Layer for user-visible error handling.

4.1.3 Subsystem interfaces

ID	Description	Inputs	Outputs
D-W1	Application → Write coordination	Domain write DTOs	Ordered write plan
D-W2	Write coordination → Integrity	Planned writes	Validation requests

Table 2: Write coordination subsystem interfaces

4.2 Integrity and constraint enforcement subsystem

This subsystem enforces **schema rules, referential integrity, and business invariants** before data reaches the query executor. It corresponds to database constraints plus application-level checks that are not expressible as single-column rules.

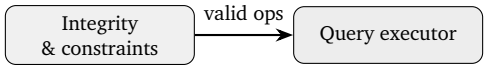

Figure 4: Integrity subsystem: validates operations before execution.

4.2.1 Assumptions

- Invalid data may arrive from bugs or malicious clients; all paths must be validated server-side.
- Constraint definitions evolve with schema migrations.

4.2.2 Responsibilities

- Reject writes that violate keys, foreign keys, or required fields.
- Apply cross-field rules (for example, ownership of a tray by the authenticated account).
- Return structured validation failures to the Application Layer.

4.2.3 Subsystem interfaces

ID	Description	Inputs	Outputs
D-I1	Write coordination → Integrity	Planned writes	Pass/fail + reasons
D-I2	Integrity → Query executor	Validated SQL ops	Executable statements

Table 3: Integrity and constraint enforcement subsystem interfaces

4.3 Query and transaction execution subsystem

The **query executor** runs SQL against PostgreSQL: transactional reads, inserts, updates, and deletes issued after validation. It returns rows, counts, or errors to the Application Layer.

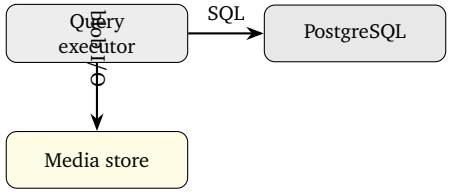

Figure 5: Query executor: issues SQL and coordinates binary I/O with storage.

4.3.1 Assumptions

- Connection pooling is managed by the Application Layer infrastructure.
- Long-running analytics queries are out of scope for interactive paths.

4.3.2 Responsibilities

- Execute parameterized statements for safety.
- Commit or roll back transactions per request scope.
- Surface deadlocks and transient errors for retry policy in higher layers.

4.3.3 Subsystem interfaces

ID	Description	Inputs	Outputs
D-Q1	Integrity → Query executor	Validated ops	Result sets / status
D-Q2	Query executor → PostgreSQL	SQL + params	Rows / ack
D-Q3	Query executor → Media store	Put/get requests	Handles / streams

Table 4: Query and transaction execution subsystem interfaces

4.4 PostgreSQL domain database subsystem

The **relational database** is the system of record for trays, plants, meshes, schedules, monitoring events, predictions, reports, and linkage to media. It satisfies SRS needs for structured historical data and dashboard queries.

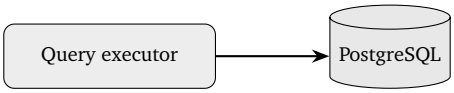

Figure 6: PostgreSQL domain database subsystem.

4.4.1 Assumptions

- Schema is versioned; migrations are applied through controlled processes.
- Backup and restore procedures meet institutional RPO/RTO.

4.4.2 Responsibilities

- Persist authoritative entities and relationships.
- Support indexed access paths for tray/plant/mesh navigation.
- Retain audit-relevant history as required by product policy.

4.4.3 Subsystem interfaces

ID	Description	Inputs	Outputs
D-P1	Query executor → PostgreSQL	SQL	Tabular results
D-P2	PostgreSQL → Query executor	Internal state	Constraints / errors

Table 5: PostgreSQL domain database subsystem interfaces

4.5 Image and media storage subsystem

Image and media storage holds camera frames, user uploads, and other large binaries. The Application Layer stores objects and retains stable references in the relational database for use in URLs served back to the Presentation Layer.

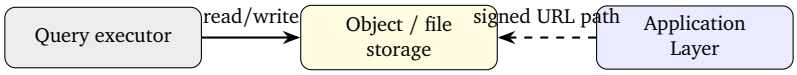

Figure 7: Image and media storage subsystem and relationship to application reads.

4.5.1 Assumptions

- Storage roots are configurable per deployment environment.
- Virus scanning or content policy, if required, occurs at integration boundaries.

4.5.2 Responsibilities

- Store and retrieve binaries keyed by application-managed paths or IDs.
- Enforce size and type limits cooperatively with validation in the Application Layer.

4.5.3 Subsystem interfaces

ID	Description	Inputs	Outputs
D-M1	Query executor → Media store	Bytes + path	Ack / handle
D-M2	Media store → Application	Stream requests	File bytes
D-M3	Application → Presentation	Media URL	Rendered imagery

Table 6: Image and media storage subsystem interfaces

5 Presentation Layer Subsystems

The Presentation Layer implements all user-visible behavior described at a high level in the SRS: authentication, dashboards, plant health display, historical views, scheduling, and mesh-oriented navigation. Each subsystem below corresponds to a coherent screen family or cross-cutting UI concern.

5.1 Authentication and session UI subsystem

This subsystem collects credentials or OAuth flows, exchanges tokens for server sessions, and guards entry to protected routes. It satisfies SRS **user authentication** expectations at the UX level.

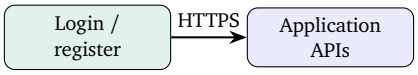

Figure 8: Authentication UI: user actions become authenticated API calls.

5.1.1 Assumptions

- Users have network connectivity for initial sign-in.
- Identity provider availability matches organizational SLA.

5.1.2 Responsibilities

- Render sign-in, registration, and sign-out affordances.
- Hold short-lived client tokens only as required by the chosen identity integration.
- Redirect unauthenticated visitors away from protected console routes.

5.1.3 Subsystem interfaces

ID	Description	Inputs	Outputs
P-A1	User → Auth UI	Credentials / OAuth	Sign-in intent
P-A2	Auth UI → Application	Session exchange requests	HTTP-only session

Table 7: Authentication and session UI subsystem interfaces

5.2 Dashboard and overview subsystem

The **dashboard** aggregates tray summaries, recent captures or predictions where available, and entry points into deeper views. It supports SRS **dashboard interface** and high-level plant status visibility.

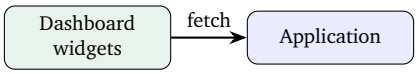

Figure 9: Dashboard subsystem requests summary aggregates from the application tier.

5.2.1 Assumptions

- Summary data volume is bounded for interactive load times.
- Chart libraries render only on supported browsers.

5.2.2 Responsibilities

- Display tray list and key health indicators.
- Link to tray detail, plants, and schedules.
- Surface empty and error states clearly.

5.2.3 Subsystem interfaces

ID	Description	Inputs	Outputs
P-D1	Dashboard → Application	Filter context	Summary DTOs
P-D2	Application → Dashboard	JSON payloads	Rendered charts / lists

Table 8: Dashboard and overview subsystem interfaces

5.3 Tray monitoring console subsystem

The **tray console** shows the active tray: latest imagery, monitoring trends, plant thumbnails, events, and optional tray-level vision uploads. It aligns with **automatic image capture** presentation and tray-scoped monitoring in the SRS.

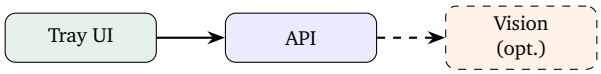

Figure 10: Tray console: drives reads and optional vision requests through the application layer.

5.3.1 Assumptions

- Each tray identifier maps to authorized user scope.
- Large images may be progressively loaded or thumbnailed.

5.3.2 Responsibilities

- Render tray imagery, charts, and plant rows.
- Submit tray-scoped uploads or vision jobs as permitted.
- Poll or refresh data according to UX policy.

5.3.3 Subsystem interfaces

ID	Description	Inputs	Outputs
P-T1	Tray UI → Application	Tray ID, actions	Tray detail DTO
P-T2	Tray UI → Application	Image multipart	Vision job result

Table 9: Tray monitoring console subsystem interfaces

5.4 Plant detail and health history subsystem

Plant detail focuses on a single plant: editable metadata, photo gallery, health reports over time, and scoped monitoring logs. It directly supports SRS **plant health status** and **historical data display**.

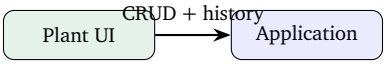

Figure 11: Plant detail subsystem: CRUD and reporting views.

5.4.1 Assumptions

- Users may edit only plants they own or that policy allows.
- Report history may be lengthy; pagination or lazy loading applies.

5.4.2 Responsibilities

- Display health timeline and latest findings.
- Support photo upload and deletion with confirmation.
- Invoke “add plant from photo” flows when enabled.

5.4.3 Subsystem interfaces

ID	Description	Inputs	Outputs
P-P1	Plant UI → Application	Plant ID	Detail + reports
P-P2	Plant UI → Application	Edits / uploads	Updated plant

Table 10: Plant detail and health history subsystem interfaces

5.5 Mesh and topology subsystem

Mesh views let users group trays for aggregated charts and cross-tray plant listings, supporting experimental layouts described in the SRS’s organizational needs without mandating a particular hardware topology.

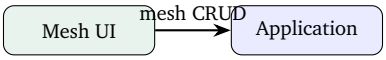

Figure 12: Mesh subsystem interacts with mesh and member tray aggregates.

5.5.1 Assumptions

- Mesh membership changes are infrequent relative to reads.

5.5.2 Responsibilities

- Create and list meshes; show merged monitoring where available.
- Navigate from mesh to constituent trays and plants.

5.5.3 Subsystem interfaces

ID	Description	Inputs	Outputs
P-M1	Mesh UI → Application	Mesh ID / payload	Mesh DTO

Table 11: Mesh and topology subsystem interfaces

5.6 Schedule management subsystem

The **schedule editor** binds capture timing to trays or meshes, supporting SRS **automatic image capture** policies at the configuration level.

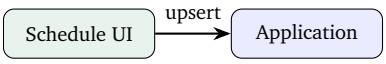

Figure 13: Schedule subsystem persists policies via application APIs.

5.6.1 Assumptions

- Schedules are stored in a timezone-aware or UTC-normalized manner in the data layer.

5.6.2 Responsibilities

- List and edit schedules with clear scope (tray vs. mesh).
- Validate user inputs (cadence, start/end) before submission.

5.6.3 Subsystem interfaces

ID	Description	Inputs	Outputs
P-S1	Schedule UI → Application	Schedule DTO	Persisted schedule

Table 12: Schedule management subsystem interfaces

6 Application Layer Subsystems

The Application Layer implements the **control plane** of AgriHome: every client request is authenticated, validated, mapped to domain operations, and translated into data-layer and optional processing-layer calls. The subsystems below follow the decomposition style of the reference architectural document while reflecting AgriHome's agronomic domain.

6.1 API entry and routing subsystem

The **API entry** subsystem exposes HTTP routes and, where used, a GraphQL endpoint. It maps URLs and methods to internal handlers and enforces global concerns such as method allow-lists and body size limits.

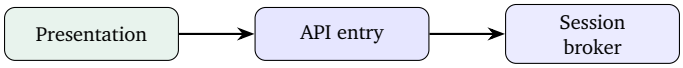

Figure 14: API entry: first server hop after the presentation layer.

6.1.1 Assumptions

- All mutating operations require authenticated context unless explicitly public.
- Rate limiting may be enforced externally (gateway) or internally per deployment.

6.1.2 Responsibilities

- Accept REST and GraphQL traffic on documented paths.
- Dispatch to validation and domain orchestration pipelines.
- Normalize HTTP status codes and error envelopes.

6.1.3 Subsystem interfaces

ID	Description	Inputs	Outputs
A-R1	Presentation → API entry	HTTP requests	Routed handler
A-R2	API entry → Session broker	Raw request	Auth context

Table 13: API entry and routing subsystem interfaces

6.2 Session and authentication broker subsystem

This subsystem verifies **server sessions** established through the identity provider flow, binds the caller to an account key used for data scoping, and rejects stale or forged cookies.

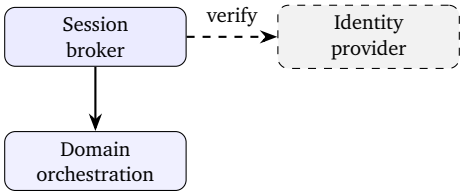

Figure 15: Session broker: establishes trusted account context for downstream work.

6.2.1 Assumptions

- Session material is transported only over TLS.
- Clock skew between servers remains within acceptable bounds for token validation.

6.2.2 Responsibilities

- Validate session cookies or bearer tokens per integration.
- Attach user identity and permissions context to the request scope.
- Support sign-out by invalidating server-side session state as implemented.

6.2.3 Subsystem interfaces

ID	Description	Inputs	Outputs
A-S1	API entry → Session broker	HTTP request	Principal or error
A-S2	Session broker → Domain	Account context	Scoped operations

Table 14: Session and authentication broker subsystem interfaces

6.3 Input validation subsystem

Input validation enforces structural correctness: JSON schema, multipart boundaries, numeric ranges for schedules, and presence of foreign keys before any database call.

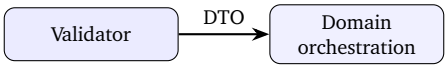

Figure 16: Validation subsystem: rejects bad input early.

6.3.1 Assumptions

- All untrusted input arrives through the API entry path.

6.3.2 Responsibilities

- Validate and sanitize payloads; coerce types where safe.
- Emit field-level errors consumable by the Presentation Layer.

6.3.3 Subsystem interfaces

ID	Description	Inputs	Outputs
A-V1	Session broker → Validator	Parsed body	Validated DTO

Table 15: Input validation subsystem interfaces

6.4 Domain orchestration subsystem

Domain orchestration implements AgriHome use cases: tray and plant CRUD, mesh membership, schedule upserts, monitoring log writes, prediction snapshots, and file handoffs. It is the primary caller of the Data Layer and the orchestrator of Processing Layer calls.

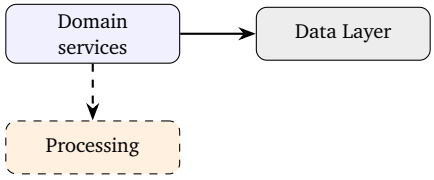

Figure 17: Domain orchestration: central business logic and integration point.

6.4.1 Assumptions

- Domain rules may evolve faster than schema; feature flags may gate behavior.

6.4.2 Responsibilities

- Implement transactional stories that satisfy SRS capabilities.
- Map database rows to API DTOs and GraphQL types.
- Coordinate retries or compensations when external services fail.

6.4.3 Subsystem interfaces

ID	Description	Inputs	Outputs
A-D1	Validator → Domain	Valid DTOs	Side effects / results
A-D2	Domain → Data Layer	SQL / storage ops	Persisted state
A-D3	Domain → Processing	Images / metadata	Inference DTOs

Table 16: Domain orchestration subsystem interfaces

6.5 GraphQL query and mutation facade subsystem

Where enabled, **GraphQL** provides a typed alternative surface for reads and selected mutations (for example mesh creation, plant updates, schedule upserts). Resolvers delegate to the same domain orchestration as REST to avoid divergent business rules.

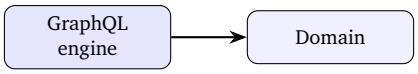

Figure 18: GraphQL facade: thin mapping to domain services.

6.5.1 Assumptions

- GraphQL complexity limits protect against abusive queries.

6.5.2 Responsibilities

- Parse, validate, and execute schema operations.
- Return typed errors suitable for client debugging.

6.5.3 Subsystem interfaces

ID	Description	Inputs	Outputs
A-G1	Presentation → GraphQL	Query/mutation	JSON result
A-G2	GraphQL → Domain	Resolver args	Domain outcomes

Table 17: GraphQL facade subsystem interfaces

6.6 Response and error handling subsystem

The **response handler** assembles payloads, sets cache headers where safe, logs structured errors, and ensures that sensitive internals are not leaked to clients—supporting SRS **security** themes at the API boundary.

Figure 19: Response handler: maps internal results to HTTP/GraphQL responses.

6.6.1 Assumptions

- Central logging infrastructure exists in deployment environments.

6.6.2 Responsibilities

- Serialize success and error bodies consistently.
- Map domain exceptions to HTTP status codes.
- Redact secrets from logs and client messages.

6.6.3 Subsystem interfaces

ID	Description	Inputs	Outputs
A-E1	Domain → Response handler	Result / exception	Outbound DTO
A-E2	Response handler → Presentation	HTTP response	Rendered UX

Table 18: Response and error handling subsystem interfaces

7 Processing Layer Subsystems

The Processing Layer encapsulates **model-based and computer-vision** work for AgriHome. It is optional in the sense that core monitoring and CRUD remain available without it; when present, it enriches the product with species hints, tray-level detections, and future scoring models aligned with the SRS's emphasis on plant health insight.

7.1 Species and leaf inference adapter subsystem

This adapter accepts a **plant or leaf image** (or derived crop) and returns a structured classification: proposed species or cultivar labels, confidence scores, and auxiliary metadata for the Application Layer to store alongside user-confirmed fields.

Figure 20: Species inference adapter: hosted model runtime may be external.

7.1.1 Assumptions

- Network latency between application and adapter is acceptable for interactive flows or runs asynchronously.
- Model versioning is tracked so historical reports can be interpreted correctly.

7.1.2 Responsibilities

- Accept a well-defined inference request contract (image bytes or URL, optional tray/plant IDs).
- Run the trained classifier or delegate to a managed inference endpoint.
- Return normalized labels and scores without persisting domain state itself.

7.1.3 Subsystem interfaces

ID	Description	Inputs	Outputs
PR-S1	Application → Species adapter	Image + context	Label JSON
PR-S2	Species adapter → Application	HTTP response	Parsed DTO

Table 19: Species and leaf inference adapter interfaces

7.2 Tray vision and layout adapter subsystem

The **tray vision** adapter analyzes a tray-level photograph to estimate plant count, regions of interest, or other layout features needed for monitoring dashboards. Outputs are merged into tray records or ephemeral UI state per product rules.

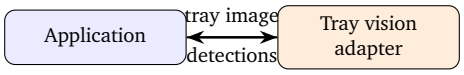

Figure 21: Tray vision adapter: round-trip inference for tray imagery.

7.2.1 Assumptions

- Images meet minimum resolution and lighting heuristics expected by the model.
- A simulator or stub may substitute in development environments.

7.2.2 Responsibilities

- Detect plants or regions; return geometry in a coordinate system agreed with the UI.
- Report confidence and failure modes explicitly.

7.2.3 Subsystem interfaces

ID	Description	Inputs	Outputs
PR-T1	Application → Tray adapter	Tray image	Detection set

Table 20: Tray vision and layout adapter interfaces

7.3 Inference normalization subsystem

Normalization converts heterogeneous model outputs into a single internal schema (units, label vocabularies, bounding-box ordering) so the Application Layer can persist results without adapter-specific branches scattered through domain code.

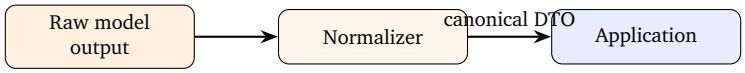

Figure 22: Normalization: single canonical shape for downstream persistence.

7.3.1 Assumptions

- Each adapter documents its native schema version.

7.3.2 Responsibilities

- Map labels to controlled vocabularies where required.
- Clamp or filter low-confidence results per policy.
- Provide stable fields for database columns and UI rendering.

7.3.3 Subsystem interfaces

ID	Description	Inputs	Outputs
PR-N1	Adapter → Normalizer	Vendor JSON	Canonical inference DTO
PR-N2	Normalizer → Application	Canonical DTO	Ready for persist

Table 21: Inference normalization subsystem interfaces

References

[1] Agrihome. *System Requirements Specification* (CSE 4316, Spring 2026). Companion requirements document.

[2] Vercel, Inc. "Next.js Documentation." `https://nextjs.org/docs` (accessed 2026).

[3] PostgreSQL Global Development Group. "PostgreSQL Documentation." `https://www.postgresql.org/docs/` (accessed 2026).

[4] Google. "Firebase Authentication." `https://firebase.google.com/docs/auth` (accessed 2026).

[5] W3C. "Progressive Web Apps." `https://www.w3.org/TR/appmanifest/` (accessed 2026).